

Share Your Knowledge

If you are an expert in your field of electronics, why not share your knowledge with the world?



Electronics For You invites electronics professionals and industry experts to write articles on their area of expertise and interest from different perspectives

Some of the topics we want you to write about are:

- Photonics: Optics & Optoelectronics
- Automation
- Power Electronics
- Components
- Telecom

So if you have an interesting topic or original article to share, please let us know at efyeditorial@efyindia.com

We would love to hear your good ideas too!

electronics
FOR YOU

which is an integer. The constants used in the code include `SLAVEADD_READ`, `SLAVEADD_WRITE` and `max200k`. `STATE` signal defines the different states of the FSM, `DATA_IN` is the input data to be written to the slave by the master controller and `DATA_OUT` is the data read from the slave by the master controller. `CLK_200kHz` is the 200kHz clock from which the SCL clock is derived. `SLAVEADD_READ` and `SLAVEADD_WRITE` are 8-bit signals comprising the 7-bit slave address and the read and the write bits, respectively appended at the end.

The first statement of the architecture body is a conditional statement which sets the SDA line to either high-impedance state 'Z' or the logic state '0' depending upon the value of `SDAINT`. SDA gets a 'Z' value if `SDAINT` is at logic '1', otherwise SDA gets logic '0' value. This is done as the SDA is an open-drain terminal and to output logic '1' on this line, we need to set this line to high-impedance state.

Let us now understand the *Output* process. If `RESET` is in logic '1' state, then the FSM is made to reset and both SCL and `SDAINT` are set to logic '1' state and the FSM goes to the state `x"00"` which is the idle state. The state `x"00"` sets both SCL and `SDAINT` to high. The next state of the FSM is the state `x"01"` in which the start condition is executed, say, `SDAINT` changes from logic '1' to logic '0' with SCL being at logic '1'. States `x"02"` and `x"03"` send the 7-bit slave address followed by the write bit. The slave address is specific to the device being addressed and is mentioned in the datasheet of the device. In our case, it is '1001000'.

When the value of `BITCOUNT` equals 8, the address bits and the write bit have been transferred and the FSM goes to state `x"12"` to get the acknowledgement from the slave. In case there is no acknowledgement from the slave, it goes to the state `x"00"` to begin the communication process again. The designer can create an error state to report the error for this situation, if desired. In case there is an acknowledgement from the slave, it goes to the

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download on source.efymag.com website.

state `x"30"`. Remember that the slave sends acknowledgement by pulling the SDA line low during the ninth clock cycle of the SCL line. States `x"30"` and `x"31"` write 8-bit data to the slave and when it is over, the FSM goes to the state `x"32"`. The data to be written is given in the `DATA_IN` signal and is specific to the application and device being used. In our case, we are writing '00000000' onto the slave device.

States `x"32"` and `x"33"` check the acknowledgement signal from the slave. In case there is no acknowledgement from the slave, the FSM goes to the state `x"00"` and when the acknowledgement is received it goes to the state `x"34"`. As mentioned before, the designer can create an error state to report the error. Next, the master controller reads the data from the slave. In states `x"40"` and `x"41"`, the master controller sends the slave address followed by the read bit. States `x"50"` and `x"51"` check the acknowledgement signal from the slave. States `x"52"` and `x"53"` read the data from the slave and move it to `DATA_OUT` signal.

In the example mentioned here, the process of sending the address along with read and write operations has been described. The designer can write the code in accordance with the design requirements of his system.

Conclusion

This part described the fundamentals of FPGA and I²C controller implementation using an FPGA. The next part will explain how to drive an LCD using an FPGA.

To be continued...

Varsha Agrawal is a scientist at Laser Science and Technology Center (LASTEC), a premier DRDO lab working in the field of laser-based defence systems. She has more than 13 years of R&D experience in the design and development of a variety of systems for defence-related applications. She has authored two books and published more than 20 research papers and technical articles